

KinTaxiBooking

Faculté des Sciences Informatiques (FASI) — Université Protestant du Congo (UPC)

L3 · Année académique 2025-2026 · Cours Laravel — Examen final

Documentation technique du projet

Étudiant : KASEREKA SALAMBUNGU SABIN

Généré le 27/06/2026 03:52

1. Description du projet

KinTaxiBooking est une application web de réservation de taxi avec suivi en temps réel, destinée au contexte de Kinshasa (RDC). Un **client** réserve une course, paie via **Mobile Money** (API Labyrinthe RDC) et suit son chauffeur sur une carte interactive. Un **chauffeur** accepte les demandes et met à jour sa position. Un **administrateur** supervise la plateforme via un tableau de bord statistique.

2. Technologies utilisées

Composant	Version / outil
PHP	8.4
Laravel	13.x
Base de données	SQLite (dev) / MySQL ou PostgreSQL (prod)
Front-end	Blade, Tailwind CSS 4, Vite 8
Temps réel	Laravel Reverb + Laravel Echo
Cartographie	Google Maps (Mapbox en alternative)
Paiement	API Labyrinthe RDC (M-Pesa, Airtel, Orange)
PDF	DomPDF (barryvdh/laravel-dompdf)
OAuth	Laravel Socialite (Google, optionnel)
API	Laravel Sanctum (tokens Bearer)
Tests	Pest 4

3. Installation pas à pas

1. Cloner le dépôt : `git clone https://github.com/Kasereka27/kin-taxi-booking-exam.git`

2. Installer les dépendances : `composer install` puis `npm install`
3. Configurer l'environnement : `cp .env.example .env` et `php artisan key:generate`
4. Créer la base SQLite : `touch database/database.sqlite` (si nécessaire)
5. Migrer et peupler : `php artisan migrate --seed`
6. Lier le stockage : `php artisan storage:link`
7. Compiler le front-end : `npm run build`
8. Lancer l'application : `composer run dev` (serveur, queue, Reverb, Vite)

Application disponible sur `http://localhost:8000` par défaut.

4. Comptes de test

Créés automatiquement par le seeder (`DatabaseSeeder`) :

Rôle	E-mail	Mot de passe
Administrateur	admin@exemple.com	password
Chauffeur	chauffeur@exemple.com	password
Client	client@exemple.com	password

5. Fonctionnalités implémentées

Niveau 1 — Fondamentaux

Obligatoire

- Architecture MVC stricte, conventions Laravel, code testé (Pest)
- Migrations, relations Eloquent, seeders réalistes
- CRUD courses (`rides`) : création, liste paginée, détail, annulation, suppression
- Recherche et filtre par statut sur l'historique
- Resource routes, routes nommées, layouts Blade réutilisables
- Validation via Form Requests, protection CSRF

Niveau 2 — Intermédiaires

Obligatoire

- Authentification multi-rôles : admin, driver, client — redirection selon le rôle
- Middleware personnalisés : `CheckRole`, `CheckAccountActive`, `RedirectIfAuthenticated`
- Dashboard administrateur : statistiques, graphique `Chart.js`, courses live, gestion utilisateurs
- Mailing (Mailables + queue) : bienvenue, vérification e-mail, réservation, statut course, paiement
- Double authentification (2FA) : OTP par e-mail, expiration configurable, activation depuis le profil

Niveau 3 — Avancées

≥ 3 requis

- API REST Sanctum — voir section 6
- Notifications Laravel (cloche in-app + e-mails métier)
- Paiement mobile Labyrinthe (initiation, callback, historique, statuts)
- Génération PDF (reçus de paiement + ce document de documentation)

- Journal d'activité admin (traçabilité connexions, courses, paiements, modération)
- Suivi temps réel WebSocket (Reverb) + carte Google Directions

6. API REST (Sanctum)

Méthode	Endpoint	Description
POST	/api/login	Obtenir un token Bearer
POST	/api/logout	Révoquer le token (auth)
GET	/api/user	Profil connecté (auth)
GET	/api/rides	Liste paginée (auth)
POST	/api/rides	Créer une course (auth, client)
GET	/api/rides/{id}	Détail (auth)
PATCH	/api/rides/{id}/cancel	Annuler (auth)
DELETE	/api/rides/{id}	Supprimer (auth)
POST	/api/labyrinthe/callback	Webhook paiement Labyrinthe

Documentation détaillée : docs/api.md

7. Schéma de base de données

Toutes les tables sont créées via migrations Laravel. Documentation complète : **docs/database-schema.md**

Table	Rôle principal
users	Comptes (admin, chauffeur, client), 2FA, statut actif
driver_profiles	Véhicule, plaque, note, position GPS, statut en ligne
rides	Courses : adresses, statut, prix, chauffeur assigné
payments	Paielements Mobile Money Labyrinthe, reçu PDF
ratings	Notes entre utilisateurs après course
otp_codes	Codes 2FA temporaires
activity_logs	Journal d'audit (connexions, actions sensibles)
notifications	Notifications in-app Laravel

8. Middleware personnalisés

Alias	Classe	Rôle
-------	--------	------

role	CheckRole	Autorise uniquement les rôles passés en paramètre de route
active	CheckAccountActive	Bloque les comptes désactivés (is_active = false)
guest	RedirectIfAuthenticated	Empêche l'accès login/register si déjà connecté

9. Difficultés rencontrées et solutions

1. **Paiement Labyrinthe en local (Windows)** — Erreurs SSL avec le bundle CA local. Solution : LABYRINTHE_VERIFY_SSL=false en développement uniquement.
2. **WebSocket Reverb** — Port 8080 occupé sur Windows. Solution : port 8090 (REVERB_PORT / REVERB_SERVER_PORT).
3. **Carte de suivi** — Passage à Google Directions API pour un itinéraire réaliste.
4. **E-mails et PDF en queue** — Mailables ShouldQueue ; file d'attente requise (composer run dev ou queue:listen).

10. Déploiement

Application non déployée en ligne au moment de la remise. Pour la production : configurer MySQL/PostgreSQL, SMTP, Reverb, exécuter `php artisan migrate --force`, `storage:link`, `npm run build`, et exposer le callback `POST /api/labyrinthe/callback`.

11. Structure du dépôt

Dossier / fichier	Contenu
app/Http/Controllers/	Contrôleurs web et API
app/Http/Middleware/	Middleware personnalisés
app/Http/Requests/	Validation des formulaires
app/Mail/ · app/Notifications/	E-mails et notifications
app/Services/	Logique métier (paiement, PDF, logs...)
database/migrations/ · seeders/	Schéma et données de test
docs/	Documentation (PDF, schéma BDD, API)
resources/views/	Vues Blade et templates PDF
routes/web.php · api.php	Routes web et API
tests/Feature/	Tests Pest